



UNIVERSITY OF STAVANGER

APPLIED ROBOT TECHNOLOGY
ELE610

Welding Simulation on ABB Robot

RS04 Documentation

Judith Zaragoza López
282308

Áron Assani
287361

2025. 02. 10.

Table of Contents

1	Aim and Objective	1
2	Identifying possible errors	1
2.1	Collision with the pegs	1
2.2	Orientation of the weld	1
3	Optimization	2
3.1	Movement and speed	2
3.2	Time constants	2
4	RobotStudio simulation	2
5	Comparison of Simulation and Real Accessibility	3
6	Simulation in Rudolf with TATEM tool	4
	Appendix	8

1 Aim and Objective

Our main goal with this project was to simulate industrial spot welding with an ABB robot and then to optimize a specific routine. In general, our goal was to make the robot as efficient as possible, with consistent results and safe margins of error. The report includes the challenges of the setup and our solutions for them.

2 Identifying possible errors

2.1 Collision with the pegs

While returning to the calibration position, the welding tool initially collided with the pegs in some configurations. This has been resolved by adjusting the safe height, so that the robot can navigate without any collisions. We changed it from the original value of -50 mm to a value of -85 mm.

2.2 Orientation of the weld

In general, the robot can reach any of the necessary angles and positions on the welding table. However, [Table 1](#) indicates the few cases in which a weld is not possible due to mechanical constraints. Since some orientations produce the same result, the problematic cases were readjusted. These tweaks resulted in a solution which is capable of all welds shown in [Table 2](#).

No.	(dx; dy)	-180°	-135°	-90°	-45°	0°	45°	90°	135°	180°
1	(0; 0)	✓	✓	✓	✓	✓	✓	✓	✗	✓
2	(60; 0)	✓	✓	✓	✓	✓	✓	✓	✗	✓
3	(120; 0)	✓	✓	✓	✓	✓	✓	✓	✗	✓
4	(180; 0)	✓	✓	✓	✓	✓	✓	✓	✗	✓
5	(0; 60)	✓	✓	✓	✓	✓	✓	✓	✗	✓
6	(60; 60)	✓	✓	✓	✓	✓	✓	✓	✗	✓
7	(120; 60)	✓	✓	✓	✓	✓	✓	✓	✗	✓
8	(180; 60)	✓	✓	✓	✓	✓	✓	✓	✗	✓
9	(0; 120)	✓	✓	✓	✓	✓	✓	✓	✗	✓
10	(60; 120)	✓	✓	✓	✓	✓	✓	✓	✗	✓
11	(120; 120)	✓	✓	✓	✓	✓	✓	✓	✗	✓
12	(180; 120)	✓	✓	✓	✓	✓	✓	✓	✗	✓
13	(0; 180)	✓	✓	✓	✓	✓	✓	✓	✗	✓
14	(60; 180)	✓	✓	✓	✓	✓	✓	✓	✗	✓
15	(120; 180)	✓	✓	✓	✓	✓	✓	✓	✗	✓
16	(180; 180)	✗	✓	✓	✓	✓	✓	✓	✗	✗

Table 1: Reachable pegs by default

3 Optimization

We made two main changes in the optimization process as we identified potential modifications to improve overall performance.

3.1 Movement and speed

To make our set routine as fast as possible, linear movements were first replaced with joint movements to improve efficiency. In addition, the speed parameters were further adjusted to reduce the total execution time. A safe height was defined to ensure clearance without compromising performance.

3.2 Time constants

Making the welds successful with the smallest possible delays was our final goal. We achieved consistent success with the following time constants:

```
VAR NUM tAdiff := 0.02;
VAR NUM tOpdelay := 0.05;
VAR NUM tRdelay := 0.025;
```

This resulted in a total waiting time of:

```
VAR num tWait := 0.495;
```

4 RobotStudio simulation

In this section, we compare the performance before and after applying speed improvements to the simulation. Initially, the default doSlow setting was set to TRUE, which slowed the execution of the commands. After changing doSlow to FALSE, we observed a notable improvement in speed. Additionally, we configured the options() setting to select the peg table and adjust the speed as needed. The following values represent the time taken for the execution of the functions before and after the changes:

Before speed improvements:

```
Time used on testPeg00() [s] = 2.667;
Time used on test4corners() [s] = 8.091;
Time used on test7pegs() [s] = 13.559;
```

After applying speed improvements:

```
Time used on testPeg00() [s] = 1.364;
Time used on test4corners() [s] = 5.252;
Time used on test7pegs() [s] = 8.716;
```

It is important to note that these values can fluctuate within a range of approximately 0.5 seconds due to factors such as the specific computer used for the tests. These variations are expected and should be considered when analysing the results.

To ensure that the robot correctly encloses the pegs, we verified its behavior by visually inspecting the simulation and using the feedback from the system's position tracking in each of the pegs by making use of the modification of `testPeg00()` we did, this way we would be able to ensure the reachability in every peg and in all rotation angles. The robot's movements were monitored to confirm that each peg was enclosed within the defined area.

Furthermore, we added a collision detection set to better observe if any collisions occurred during the robot's movements. By monitoring the output signals from the simulation, we ensured that no collisions took place at any point in the process. This allowed us to verify that the robot's path remained clear and accurate as it enclosed the pegs.

5 Comparison of Simulation and Real Accessibility

In this section, we present a comparison between theoretical accessibility, calculated based on the parameters defined in the `wobjTestBoardOnTable` function, and real accessibility, which was obtained from simulation on the actual robot.

The theoretical accessibility is based on ideal values of orientation and position, while the real accessibility reflects the results from the real robot. Through this comparison, we aim to analyze the differences between the expected and observed results, with the goal of identifying critical areas for improvement in the system's accuracy and performance.

The reachability with regard to the pegs is the same as in the simulation, based on the results obtained from testing on the actual robot and the modifications applied to reach all pegs shown in [Table 2](#).

No.	(dx; dy)	-180°	-135°	-90°	-45°	0°	45°	90°	135°	180°
1	(0; 0)	✓	✓	✓	✓	✓	✓	✓	✓	✓
2	(60; 0)	✓	✓	✓	✓	✓	✓	✓	✓	✓
3	(120; 0)	✓	✓	✓	✓	✓	✓	✓	✓	✓
4	(180; 0)	✓	✓	✓	✓	✓	✓	✓	✓	✓
5	(0; 60)	✓	✓	✓	✓	✓	✓	✓	✓	✓
6	(60; 60)	✓	✓	✓	✓	✓	✓	✓	✓	✓
7	(120; 60)	✓	✓	✓	✓	✓	✓	✓	✓	✓
8	(180; 60)	✓	✓	✓	✓	✓	✓	✓	✓	✓
9	(0; 120)	✓	✓	✓	✓	✓	✓	✓	✓	✓
10	(60; 120)	✓	✓	✓	✓	✓	✓	✓	✓	✓
11	(120; 120)	✓	✓	✓	✓	✓	✓	✓	✓	✓
12	(180; 120)	✓	✓	✓	✓	✓	✓	✓	✓	✓
13	(0; 180)	✓	✓	✓	✓	✓	✓	✓	✓	✓
14	(60; 180)	✓	✓	✓	✓	✓	✓	✓	✓	✓
15	(120; 180)	✓	✓	✓	✓	✓	✓	✓	✓	✓
16	(180; 180)	✓	✓	✓	✓	✓	✓	✓	✓	✓

Table 2: Reachable pegs with the modifications

We also implemented the same instructions on manual mode, except `test4pegs()`, to see the actual performance on the robot and the time it takes for it to do the processes, this way we would have a comparison between simulation and the actual robot. We had as a result the following values:

Implementation in the real ABB robot in manual mode:

Time used on `testPeg00()` [s] = 16.347;

Time used on `test7pegs()` [s] = 48.603;

In automatic mode, we only tested `test7pegs()` twice to be able to take the report, and we ended with a similar result as the simulation was showing in the first place:

Implementation in the real ABB robot in automatic mode:

Time used on `test7pegs()` [s] = 8.793;

Time used on `test7pegs()` [s] = 8.316;

6 Simulation in Rudolf with TATEM tool

After checking the correct performance of the robot, we used the automatic mode and exported the report using the TATEM tool mounted on the welding tool. With that, we were able to take the data of the successes we had and the json files to make a proper visualization of the whole performance.

In the first simulation, we reached an error in which we realized that we had not set the values for `tA`, `tO`, and `tR`, which were necessary for the robot to perform properly, so we had a failure as response as seen in [Figure 1](#).

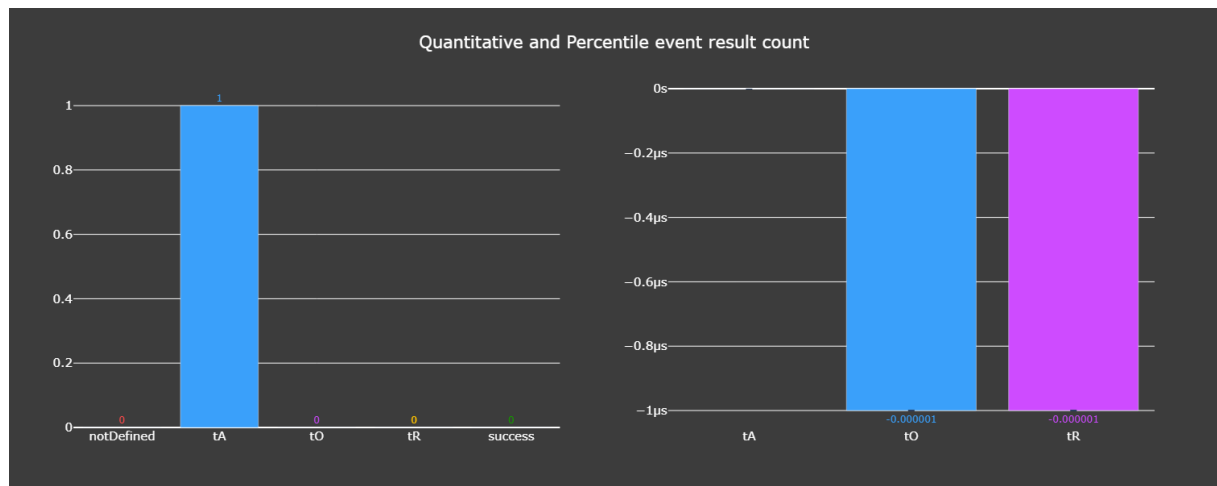


Figure 1: testPeg00 Failure

For visualization of results with individual welds, the error we had as a result gave us the table from [Figure 2](#)

no	eventStart	eventEnd	eventDuration	operationStart	doOff	operationDuration	tA	tO	tR	eventResult
0	528214779	-1	532514788	0	0	0	0	-1	-1	TaError

Figure 2: testPeg00 Failure Table

After including the pre-setted values of the constants, we had the failure on, we got the result shown in Figure 3.

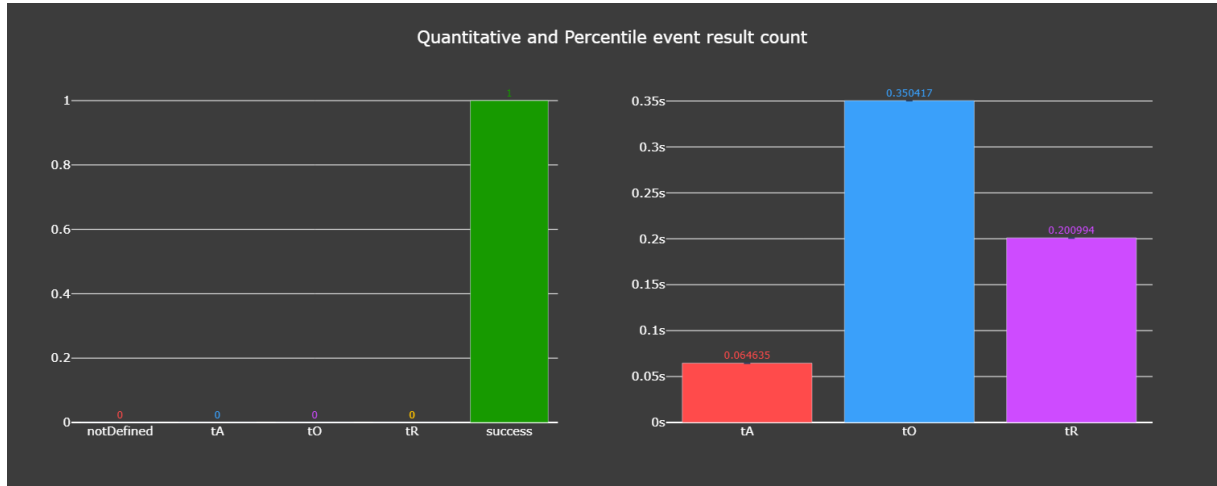


Figure 3: testPeg00 Success

no	eventStart	eventEnd	eventDuration	operationStart	doOff	operationDuration	tA	tO	tR	eventResult
0	874732880	875384463	651583	874833052	875183469	350417	64635	350417	200994	Success

Figure 4: testPeg00 Success Table

After verifying that testPeg00 worked correctly in other pegs and different orientations, we proceeded with the other instructions.

In the following tests, we evaluated the performance of different configurations. The results of test4corners can be seen in Figure 5, Figure 6, while the results of test7pegs are shown in Figure 7 and Figure 8.

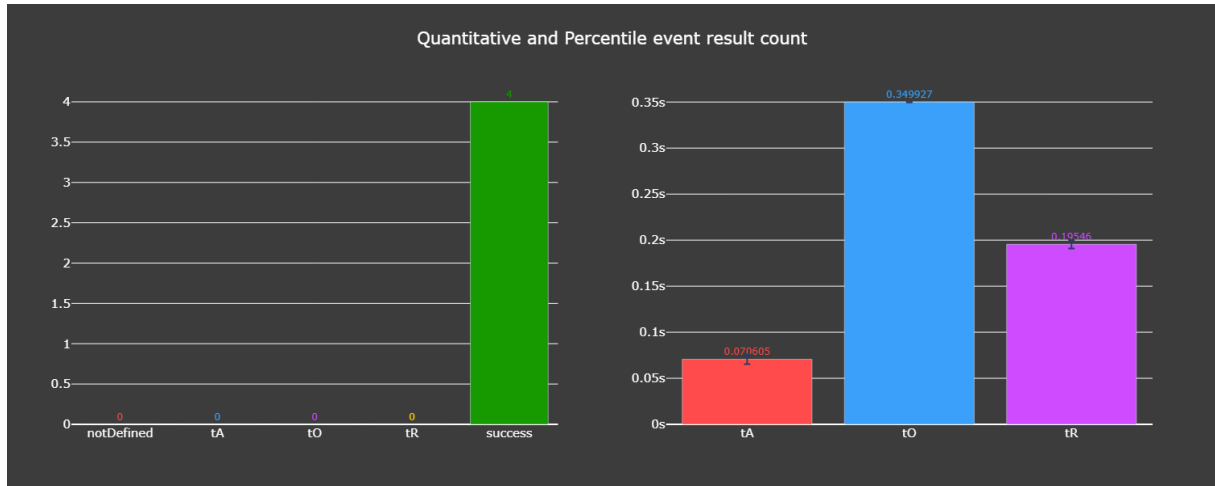


Figure 5: test4corners Success

no	eventStart	eventEnd	eventDuration	operationStart	doOff	operationDuration	tA	tO	tR	eventResult
0	1899453971	1900101224	647253	1899534442	18	1899534424	null	349790	197032	Success
1	1911251645	1911905580	652935	1911351646	1911702143	250497	62530	350497	203437	Success
2	1914623738	1915266316	642378	1914724146	1915073868	349722	70131	349722	192448	Success
3	1917930553	1918574932	644379	1918030995	1918380877	349882	73797	349882	194055	Success

Figure 6: test4corners Success Table

In [Figure 7](#) and [Figure 8](#) we display 21 results as we had a problem where the JSON file was not loaded correctly on the program. Despite that, in the end, we managed to extract the last 3 runs of test7pegs resulting with the graphs and table of 21 records.

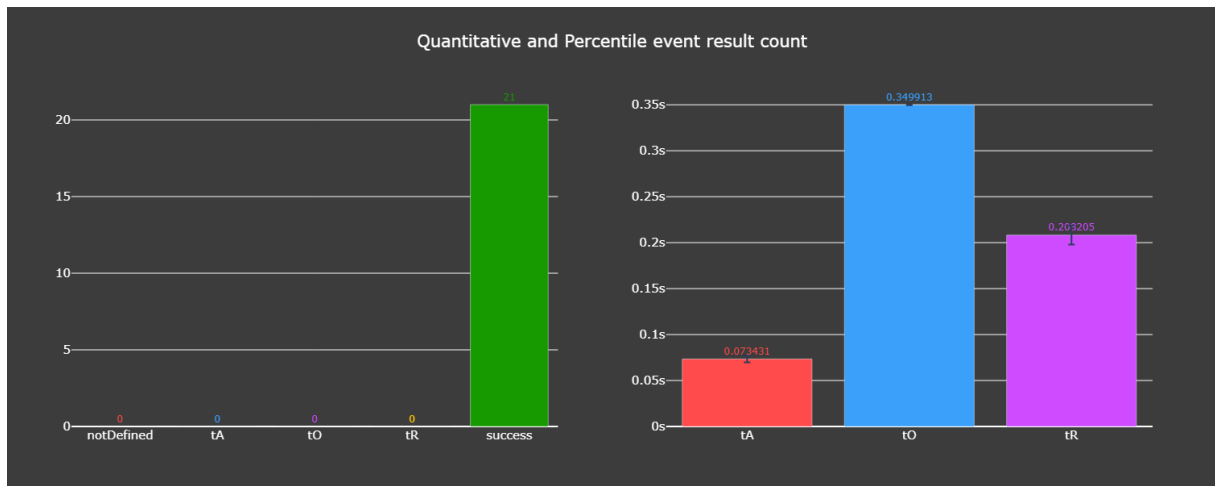


Figure 7: test7pegs Success

no	eventStart	eventEnd	eventDuration	operationStart	doOff	operationDuration	tA	tO	tR	eventResult
0	-1522199969	-1521569226	630743	-1522099877	-1521749801	350078	79256	350076	180575	Success
1	-1506900004	-1506263879	636125	-1506799988	-1506450245	349743	76331	349743	186366	Success
2	-1500744575	-1500098833	646742	-1500644086	-1500294408	349678	60507	349678	198575	Success
3	-1497186630	-1496542233	644397	-1497086122	-1496736515	349607	70224	349607	194282	Success
4	-1489405892	-1488756853	649039	-1489305573	-1488955737	349636	69601	349636	198884	Success
5	-1485041978	-1484393138	648840	-1484941479	-1484591693	349786	64926	349786	198555	Success
6	-1480428930	-1479788269	646661	-1480328439	-1479978976	349463	67985	349463	190707	Success
7	-852625865	-851978826	647039	-852525831	-852175674	350157	80256	350157	196848	Success
8	-850941907	-850288254	653653	-850841513	-850491762	349751	75792	349751	203508	Success
9	-849800941	-849124407	676534	-849700580	-849350771	349809	70266	349809	226364	Success
10	-848777971	-848120989	656982	-848677891	-848327817	350074	74373	350074	206828	Success
11	-847838004	-846864372	679632	-847437849	-847087240	350909	73622	350909	222968	Success
12	-846494038	-845831922	662116	-846393995	-846043882	350114	73557	350113	211960	Success
13	-845435074	-844777652	657422	-845335019	-844984910	350109	74094	350109	207258	Success
14	-826700580	-826049796	656784	-826600145	-826250428	349717	79949	349717	200632	Success
15	-825011616	-824362352	649264	-824911134	-824561485	349649	75708	349649	199133	Success
16	-823874615	-823202512	672103	-823774479	-823424523	349596	70176	349596	222011	Success
17	-822855712	-822199104	656608	-822755711	-822405516	350195	74231	350195	206412	Success
18	-821615747	-820946481	669266	-821515489	-821165191	350298	73595	350298	218710	Success
19	-820575629	-819913963	661666	-820475210	-820125615	349993	73155	349595	211652	Success
20	-819516364	-818855631	666733	-819416084	-819066384	349700	73538	349700	210753	Success

Figure 8: test7pegs Success Table

```

MODULE MainModule

! made by KS November 2023

! (based on how RWS and JAMR adapted template program from ABB spring 2023)

!

! main module for simulation using TATEM tool and a test board with pegs attached

! position of tool point is relative to tool0, translated so the Tool Center Point (TCP) is
! between the two fingers at level of the gap for the front LED, it is rotated -60 degrees around x-axis to make z-axis
! point 'forward' from the tool, the 180 degrees around the z-axis so y-axis points into the widest finger,
! and x-axis points to the 'right'.

! PERS tooldata TatemTool1:=[TRUE,[[0,-48.3,153.5],[0,0,-0.5,0.866025404]],[0.3,[0,0,1],[1,0,0,0],0,0,0]];
PERS tooldata TatemTool1:=[TRUE,[[0,-48.3,153.5],[0,0,-0.5,0.866025404]],[0.3,[0,0,1],[1,0,0,0],0,0,0]];

! the center of the table (not used), relative to robot coordinate system (not world)
TASK PERS wobjdata wobjTableR:=[FALSE,TRUE,"",[[150,500,8],[1,0,0,0]],[[0,0,0],[1,0,0,0]]];

! the work object used with test board on table
!
! x y z
TASK PERS wobjdata wobjTestBoardOnTable :=
[FALSE,TRUE,"",[[0,544,146],[0.965928085,0.258810616,0,0]],[[0,0,0],[1,0,0,0]]];

! the one used with test board on conveyer belt
!
! x y z
TASK PERS wobjdata wobjTestBoardOnBelt := [FALSE,TRUE,"",[[599.763,-318.75,282.622],[0.265084,0.0713605,-0.252646,-
0.927797]],[[0,0,0],[1,0,0,0]]];

! the work object to use or used last time
TASK PERS wobjdata wobjTestBoard := [FALSE,TRUE,"",[[0,544,146],[0.965928,0.258811,0,0]],[[0,0,0],[1,0,0,0]]];

! speeddata that are fixed
CONST speeddata vVeryFast := [5000,500,5000,1000];
CONST speeddata vFast := [1000,500,5000,1000];
CONST speeddata vSlow := [200,500,5000,1000];
CONST speeddata vVerySlow := [100,500,5000,1000];

VAR NUM tA := 0.1; ! The time it takes to activate the tool

VAR NUM tAdelay := 0.08; ! The amount of time before entering the point that the DO signal will be turned on
(should be less than tA) !! CHANGED FROM 0.06 to 0.08

VAR NUM tAdiff := 0.02; ! Amount of time the robot will stand still in its position before executing the
spotwelding (MUST be tA-tA_Delay) !! CHANGED FROM 0.04 to 0.02

! note that t0 is a reserved word, i.e. T0 as used in FOR-loops, and gives error when used otherwise

! RAPID identifiers (names) are case-insensitive, see Rapid Overview (on Help) Basic RAPID programming - Program
structure - Basic elements

VAR NUM tOp := 0.3; ! The execution time of the task

VAR NUM tOpdelay := 0.05; ! Extra time to make sure the task is finished before retracting the tool !! CHANGED
FROM 0.1 to 0.05

VAR NUM tR := 0.1; ! The time it takes to retract the tool

VAR NUM tRdelay := 0.025; ! Extra time to make sure that the tool has been retracted before exiting the point
!! CHANGED FROM 0.05 to 0.025

VAR num tWait := 0.495; ! tWait, == (tAdiff + tOp + tOpdelay + tR + tRdelay) !! CHANGED FROM 0.59 to 0.495

CONST robtarget AboveTestBoard:= [[90,90,100],[0,1,0,0],[1,0,-1,1],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

```

```

! pegs at the 4 corners of the test board

CONST num nPegs := 16; ! number of different pegs

CONST robtarget Peg00:=[[0,0,0],[0,1,0,0],[1,0,-1,1],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

CONST robtarget Peg03:=[[0,180,0],[0,1,0,0],[1,0,-1,1],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

CONST robtarget Peg30:=[[180,0,0],[0,1,0,0],[0,-1,0,1],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

CONST robtarget Peg33:=[[180,180,0],[0,1,0,0],[0,-1,0,1],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];


VAR num row;

VAR num col;


VAR intnum igun_on;

VAR trigdata PGunOn;

CONST jointtarget jCalibPosL:=[[60,30,0,0,-10,90],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

CONST jointtarget jCalibPos0:=[[0,0,0,0,0,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

CONST jointtarget jCalibPosR:=[[-15,15,0,0,-10,90],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

VAR jointtarget jCalibPos;


VAR bool doSlow := FALSE;

VAR bool useFlexPendant := TRUE;

VAR bool useBoardOnTable := TRUE;

VAR num rows{4} := [0, 60, 120, 180];

VAR num cols{4} := [0, 60, 120, 180];


PROC main()

VAR num peg2Test; VAR num rot_z;


MoveAbsJ jCalibPos0,v1000,z10,tool0;

IF useBoardOnTable THEN

wobjTestBoard := wobjTestBoardOnTable;

jCalibPos := jCalibPosL;

ELSE

wobjTestBoard := wobjTestBoardOnBelt;

jCalibPos := jCalibPosR;

ENDIF


IF useFlexPendant THEN

TPERase;

reg1 := 0;

ELSE

reg1 := 2;

ENDIF


! While loop to choose the task

```

```

WHILE reg1 < 5 DO
    TEST reg1
    CASE 1:
        TPreadNum peg2Test, "Which peg do you want to test? (1 - 16):";
        IF ((peg2Test >= 1) AND (peg2Test <= 16)) THEN
            TPreadNum rot_z, "Add a rotation angle (-180 - +180 in 45º steps):";
            IF (((rot_z MOD 45) = 0) AND (-180 <= rot_z) AND (rot_z <= 180)) THEN
                IF rot_z = 135 THEN
                    rot_z := -45;
                ELSEIF peg2Test = 16 AND Abs(rot_z) = 180 THEN
                    rot_z := 0;
                ENDIF
                testPeg peg2Test, rot_z;
            ELSE
                TPWrite "Invalid number, it must be between -180 and 180 and in 45º steps.";
            ENDIF
        ELSE
            TPWrite "Invalid number, it must be between 1 and 16.";
        ENDIF
    CASE 2:
        test4corners;
    CASE 3:
        test7pegs;
    CASE 4:
        options;
    CASE 5:
        TPWrite "Exiting the program";
    DEFAULT:
        WaitTime 0.1;
    ENDTEST
    IF useFlexPendant THEN
        TPreadFK reg1, "Select a task:",
            "Test a peg",
            "Test 4 corners",
            "Test 7 pegs",
            "Options",
            "Quit";
    ELSE
        reg1 := 5;
    ENDIF
ENDWHILE

MoveAbsJ jCalibPos0,v1000,z10,too10;

ENDPROC

```

```

FUNC string mapUserChoiceToGrid (num userChoice)

    VAR string value;

    VAR string row_string;

    VAR string col_string;

    col := (userChoice - 1) DIV 4; ! X Value

    row := (userChoice - 1) MOD 4; ! Y Value

    col_string := ValToStr(col);

    row_string := ValToStr(row);

    value := col_string + row_string;

    RETURN value;

ENDFUNC

FUNC num GetPegPosition(num gridPos)

    VAR num position;

    ! Compute the position value

    position := 60 * gridPos;

    RETURN position;

ENDFUNC

PROC testPeg (num pegIndex, num rot_z)

    VAR num numTime := 0;

    VAR string gridPos; VAR string gridPos2;

    VAR bool a; VAR bool b;

    VAR robtarget newPeg;

    VAR num pos_x; VAR num pos_y;

    IF useFlexPendant THEN

        TPWrite "test4corners() started";

        WHILE ((1 <= pegIndex) AND (pegIndex <= 16) AND ((rot_z MOD 45) = 0) AND (-180 <= rot_z) AND (rot_z <= 180)) DO

            IF rot_z = 135 THEN

                rot_z := -45;

            ENDIF

            gridPos := mapUserChoiceToGrid(pegIndex);

            gridPos2 := StrPart(gridPos, 1, 1);

            a := StrToVal(gridPos2, col);

            gridPos2 := StrPart(gridPos, 2, 1);

            b := StrToVal(gridPos2, row);

            MoveAbsJ jCalibPos,vFast,z10,tool0;

            ClkReset clock1;

```

```

        ClkStart clock1;

        initTatemTool;

        pos_x := GetPegPosition(col);
        pos_y := GetPegPosition(row);
        IF pos_y = 0 THEN
            rot_z := 180;
        ENDIF
        doPeg00 pos_x, pos_y, rot_z;

        MoveAbsJ jCalibPos,vFast,z10,tool0;
        numTime := ClkRead(clock1);
        TPWrite "Time used on testPeg() [s] = " \Num:=numTime;
        TPreadNum pegIndex, "Which peg do you want to test? (1 - 16):";
        IF ((1 <= pegIndex) AND (pegIndex <= 16)) THEN
            TPreadNum rot_z, "Add a rotation angle (-180 - +180 in 45° steps):";
        ENDIF
    ENDWHILE
ELSE
    TPWrite "Invalid value. Exiting the task.";
    RETURN;
ENDIF
ENDPROC

PROC test4corners()
    VAR num numTime := 0;

    IF useFlexPendant THEN
        TPWrite "test4corners() started";
    ENDIF
    MoveAbsJ jCalibPos,vFast,z10,tool0;
    ClkReset clock1;
    ClkStart clock1;

    initTatemTool;
    doPeg00 0, 0, 180;
    doPeg00 180, 0, 0;
    doPeg00 180, 180, 0;
    doPeg00 0, 180, 0;

    MoveAbsJ jCalibPos,vFast,z10,tool0;
    numTime := ClkRead(clock1);
    IF useFlexPendant THEN
        TPWrite "Time used on test4corners() [s] = " \Num:=numTime;
    
```

```

ENDIF
ENDPROC

PROC test7pegs()
    VAR num numTime := 0;

    IF useFlexPendant THEN
        TPWrite "test7pegs() started";
    ENDIF

    MoveAbsJ jCalibPos,vFast,z10,tool0;
    ClkReset clock1;
    ClkStart clock1;

    initTatemTool;
    doPeg00 0, 180, 90;
    doPeg00 0, 0, 180;
    doPeg00 180, 0, -90;
    doPeg00 60, 180, -45;
    doPeg00 60, 60, 45;
    doPeg00 120, 120, 0;
    doPeg00 180, 120, -45;

    MoveAbsJ jCalibPos,vVeryFast,z10,tool0;
    numTime := ClkRead(clock1);
    IF useFlexPendant THEN
        TPWrite "Time used on test7pegs() [s] = " \Num:=numTime;
    ENDIF
ENDPROC

PROC options()
    VAR num useBoardOnTable_userOption := 2;
    VAR num doSlow_userOption := 2;

    IF useFlexPendant THEN
        TPWrite "options() started";
    ENDIF

    WHILE ((useBoardOnTable_userOption <> 0) AND (useBoardOnTable_userOption <> 1)) DO
        TPReadNum useBoardOnTable_userOption, "Do you want to use the Board on the table? (1 True, 0 False):";
    ENDWHILE

    IF useBoardOnTable_userOption = 1 THEN
        useBoardOnTable := TRUE;
    ELSEIF useBoardOnTable_userOption = 0 THEN
        useBoardOnTable := FALSE;

```

```

ENDIF

    WHILE (doSlow_userOption <> 1 AND doSlow_userOption <> 0) DO
        TPreReadNum doSlow_userOption, "Do you want to go fast or slow? (1 Slow, 0 Fast):";
    ENDWHILE

    IF doSlow_userOption = 1 THEN
        doSlow := TRUE;
    ELSEIF doSlow_userOption = 0 THEN
        doSlow := FALSE;
    ENDIF

ENDPROC

PROC doPeg00(num dx, num dy, num rotz)
    ! arguments here should be given relative to work object
    ! but RelTool adjust position relative to tool coordinate system
    ! Below sign of dx is kept, sign of dy (and dz) and rotation around z-axis are reversed,
    ! this will, for the cases here, make arguments dx, dy, and rotz as if they were
    ! related to the work object.
    !
    VAR num t1 := 0.025;    ! at peg wait t1 and the activate tool
    VAR num t2 := 0.450;    ! then wait t2, staying calm on peg
    VAR num t3 := 0.125;    ! deactivate tool, and wait t3 until moving from peg
    !
    IF doSlow THEN
        MoveJ RelTool(Peg00, dx, -dy, -50, \Rz:= -rotz), vVeryFast, z5, TatemTool1\WObj:=wobjTestBoard;
        MoveJ RelTool(Peg00, dx, -dy, 0, \Rz:= -rotz), vSlow, fine, TatemTool1\WObj:=wobjTestBoard;
        WaitTime t1;
        SetDO AirValve, 1; ! activate tool
        WaitTime t2;
        SetDO AirValve, 0; ! deactivate tool
        WaitTime t3;
        MoveJ RelTool(Peg00, dx, -dy, -85, \Rz:= -rotz), vSlow, z5, TatemTool1\WObj:=wobjTestBoard;
    ELSE
        ! Here try to do a faster 'weld simulation', using TriggL
        ! Moving to above wanted position (dx, dy) from Peg00 and tool rotated rotz degrees clockwise
        MoveJ RelTool(Peg00, dx, -dy, -85, \Rz:= -rotz), vVeryFast, z150, TatemTool1\WObj:=wobjTestBoard;
        MoveJ RelTool(Peg00, dx, -dy, -50, \Rz:= -rotz), vFast, z150, TatemTool1\WObj:=wobjTestBoard;
        ! The signal is turned on tAdelay second before (=above) the target point
        TriggJ RelTool(Peg00, dx, -dy, 0, \Rz:= -rotz), vFast, PGunOn, fine, TatemTool1\WObj:=wobjTestBoard;
        WaitTime tWait;    ! should be the minimum time to wait
        ! Move up again
    ENDIF
ENDPROC

```



```

        MoveJ RelTool(Peg00, dx, -dy, -50, \Rz:= -rotz), vFast, z150, TatemTool1\WObj:=wobjTestBoard;

        MoveJ RelTool(Peg00, dx, -dy, -85, \Rz:= -rotz), vVeryFast, z150, TatemTool1\WObj:=wobjTestBoard;

    ENDIF

ENDPROC

! The last two functions for using trigger to activate and and interrupt to deactivate tool

PROC initTatemTool()

    ! just return IF doSlow

    IF NOT doSlow THEN

        ! initialize the TATEM tool, for using trigger and interrupt

        ! Connect the triggdata variable PGunOn to the DO signal AirValve

        ! and set the startup time of the tool as tA_delay seconds before reaching the point

        ! the last argument, 1, is the value to assign to the DO signal when triggered.

        TriggIO PGunOn, tAdelay\Time \Dop:=AirValve, 1;

        IDelete igun_on;

        CONNECT igun_on WITH resetSignal;

        ISignalDO AirValve, 1, igun_on;

    ENDIF

ENDPROC

TRAP resetSignal

    TEST INTNO

    CASE igun_on:

        ISleep igun_on;

        SetDO \SDelay:=tA+tOp+tOpdelay, AirValve, 0; ! Triggering DO off

        IWatch igun_on;

    ENDTEST

ENDTRAP

ENDMODULE

```